

ACTIVIDAD 11: VALIDATION OF START OF PATH

PROGRAMACION WAP

JESÚS ISRAEL MARTÍNEZ ESCOBEDO

Contenido

1. NIVEL.....	2
2. TIPO DE EXPLOTACIÓN.....	2
3. OBJETIVO DEL LABORATORIO	2
4. EXPLICACIÓN TÉCNICA DE LA VULNERABILIDAD	2
5. PROCEDIMIENTO PASO A PASO	2
5.1 Acceso al laboratorio	3
5.2 Identificación de la solicitud	3
5.3 Envío de la solicitud a Repeater.....	3
5.4 Modificación del parámetro vulnerable	3
5.5 Envío de la solicitud modificada	4
5.6 Análisis de la respuesta.....	4
5.7 Confirmación de la resolución	4
6. EVIDENCIAS.....	4
7. EXPLICACIÓN DEL PAYLOAD	7
8. MITIGACIÓN RECOMENDADA.....	8
CONCLUSIÓN	8

1. NIVEL

Practitioner

2. TIPO DE EXPLOTACIÓN

File Path Traversal mediante bypass de validación del inicio de la ruta.

3. OBJETIVO DEL LABORATORIO

El objetivo del laboratorio es identificar y explotar una vulnerabilidad de **File Path Traversal** en una aplicación web que valida que la ruta proporcionada mediante el parámetro filename comience con el directorio esperado.

La finalidad consiste en aprovechar esta validación para acceder a un archivo ubicado fuera del directorio autorizado y recuperar el contenido de:

`/etc/passwd`

4. EXPLICACIÓN TÉCNICA DE LA VULNERABILIDAD

El laboratorio presenta una vulnerabilidad de **File Path Traversal** en la funcionalidad utilizada para mostrar las imágenes de los productos.

La aplicación transmite mediante el parámetro filename la ruta completa del archivo que debe recuperar y valida que la ruta proporcionada comience con el directorio esperado:

`/var/www/images/`

El problema se presenta debido a que la validación comprueba únicamente el **inicio de la ruta**, sin verificar correctamente cuál será la ubicación final del archivo después de resolver las secuencias de recorrido `../`.

Esto permite construir una ruta que inicialmente comienza con el directorio autorizado, pero que posteriormente utiliza secuencias de recorrido para abandonar dicho directorio.

De esta manera, una ruta como:

`/var/www/images/../../etc/passwd`

puede superar una validación que únicamente compruebe el prefijo `/var/www/images/`.

Una vez procesadas las secuencias `../`, la ruta puede salir del directorio destinado a las imágenes y alcanzar el archivo `/etc/passwd`.

La vulnerabilidad demuestra que validar solamente el inicio de una ruta no es suficiente para impedir un ataque de Path Traversal, ya que la ruta debe analizarse después de ser normalizada.

5. PROCEDIMIENTO PASO A PASO

5.1 Acceso al laboratorio

Se accedió al laboratorio de File Path Traversal, validation of start of path de PortSwigger Web Security Academy.

La funcionalidad vulnerable se encuentra en la solicitud utilizada para obtener las imágenes de los productos.

5.2 Identificación de la solicitud

Se utilizó **Burp Suite** para interceptar la solicitud generada por la aplicación al cargar una imagen de producto.

La solicitud original utilizaba el parámetro filename con una ruta correspondiente al directorio de imágenes:

```
GET /image?filename=/var/www/images/61.jpg HTTP/2
```

La ruta contenía el directorio esperado por la aplicación:

```
/var/www/images/
```

5.3 Envío de la solicitud a Repeater

La solicitud interceptada fue enviada a **Burp Suite Repeater**, donde fue posible analizar y modificar manualmente el parámetro filename.

5.4 Modificación del parámetro vulnerable

Una vez en Burp Suite Repeater, se modificó manualmente el parámetro filename.

El valor original:

```
/var/www/images/61.jpg
```

fue sustituido por:

```
/var/www/images/../../etc/passwd
```

La solicitud modificada quedó de la siguiente manera:

```
GET /image?filename=/var/www/images/../../etc/passwd HTTP/2
```

El objetivo fue conservar el prefijo `/var/www/images/` que la aplicación esperaba encontrar al inicio de la ruta y agregar posteriormente las secuencias de recorrido necesarias para acceder al archivo ubicado fuera de dicho directorio.

5.5 Envío de la solicitud modificada

Después de introducir el payload, se envió la solicitud desde **Burp Suite Repeater**.

El servidor respondió correctamente a la petición y devolvió una respuesta HTTP con código:

HTTP/2 200 OK

El cuerpo de la respuesta contenía información correspondiente al archivo `/etc/passwd`.

5.6 Análisis de la respuesta

El cuerpo de la respuesta contenía el contenido del archivo `/etc/passwd`, comenzando con la entrada correspondiente al usuario `root`.

```
root:x:0:0:root:/root:/bin/bash
```

También se observaron otras cuentas del sistema, confirmando que la aplicación había permitido acceder al archivo solicitado.

5.7 Confirmación de la resolución

Después de ejecutar correctamente la explotación, la plataforma **PortSwigger Web Security Academy** mostró la confirmación:

Lab Solved

Esta confirmación indica que la plataforma reconoció correctamente la recuperación del archivo solicitado y que el objetivo establecido para el laboratorio fue alcanzado.

6. EVIDENCIAS

Figura 1. Solicitud HTTP original.

Descripción

La captura muestra la solicitud HTTP original utilizada para recuperar la imagen del producto. El parámetro `filename` contiene una ruta dentro del directorio `/var/www/images/`, que corresponde a la ubicación esperada por la aplicación.

Evidencia:

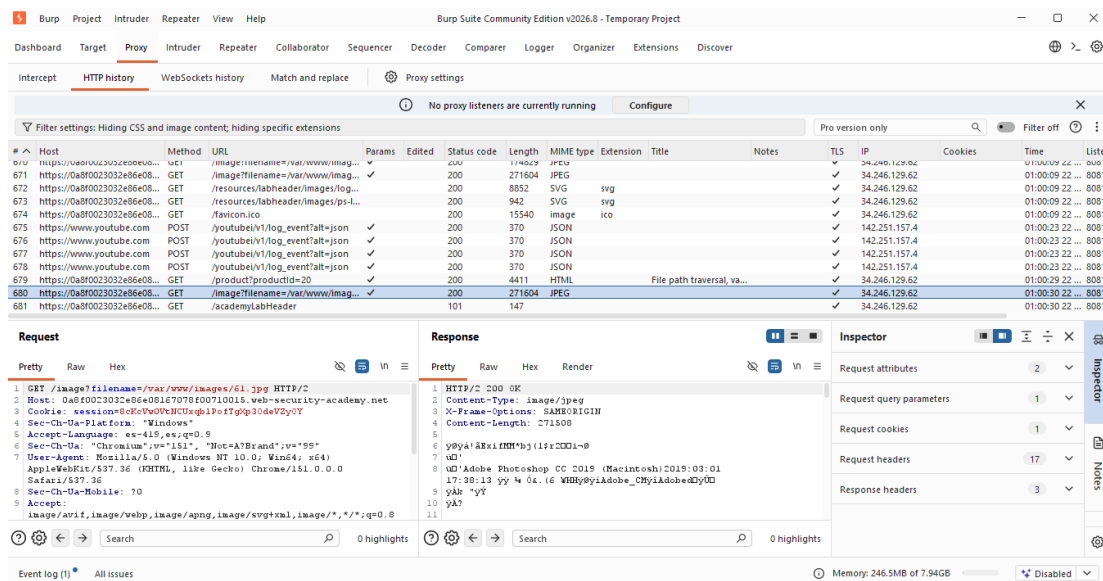


Figura 2. Payload utilizado

Descripción:

La captura muestra la solicitud modificada mediante Burp Suite. El parámetro filename conserva el prefijo /var/www/images/, pero posteriormente incorpora ../../../../ para realizar el recorrido hacia directorios superiores y acceder al archivo /etc/passwd.

Evidencia:

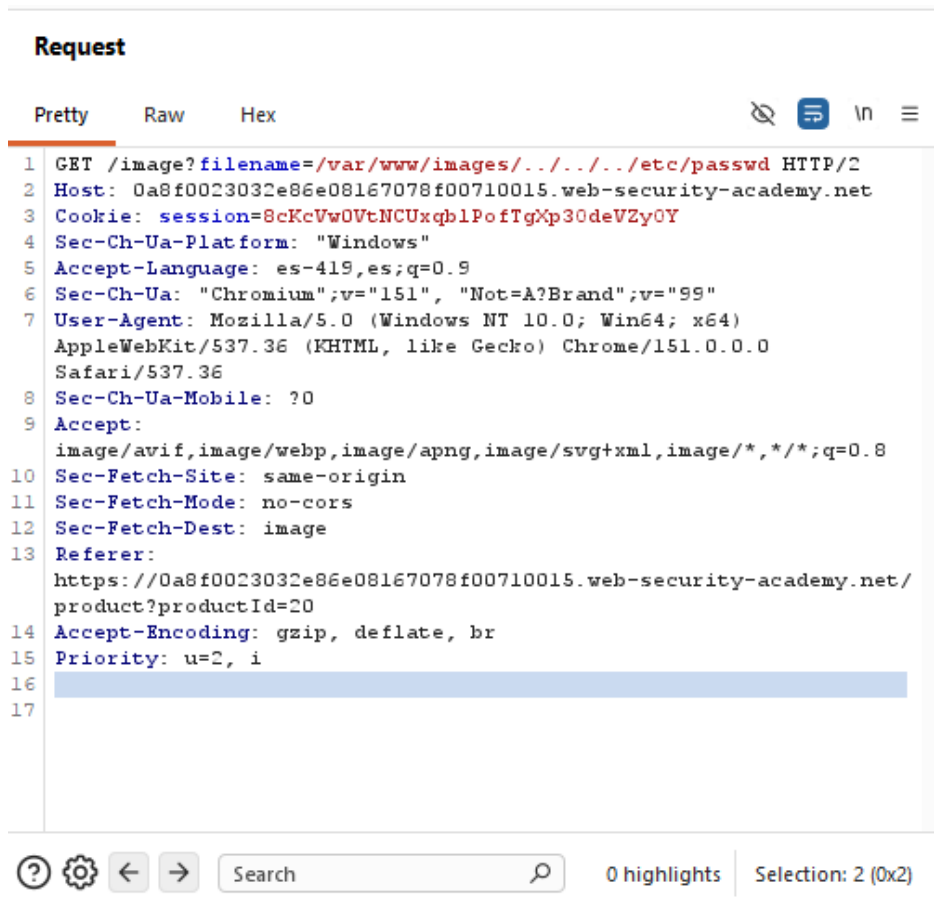
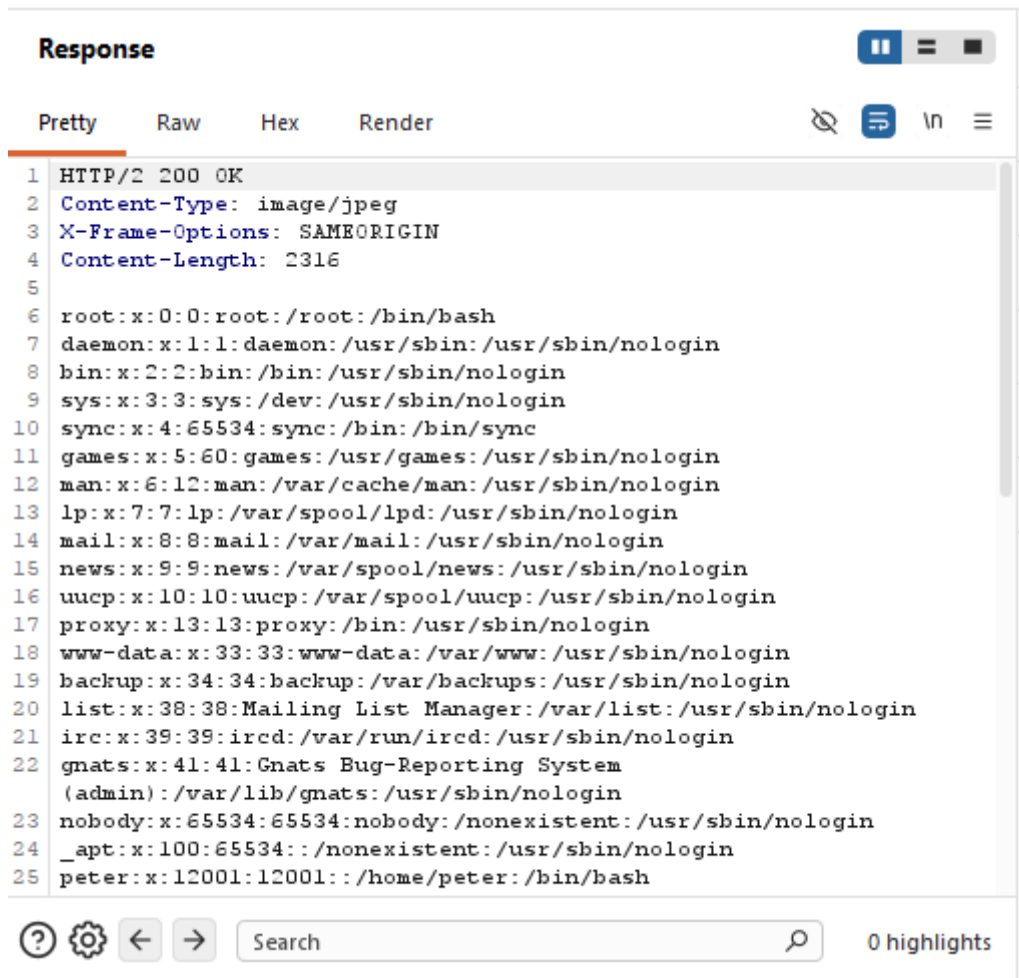


Figura 3. Respuesta vulnerable

Descripción:

La captura muestra la respuesta del servidor después de enviar la solicitud modificada. El cuerpo de la respuesta contiene entradas correspondientes al archivo `/etc/passwd`, demostrando que fue posible acceder a un archivo fuera del directorio destinado a las imágenes.

Evidencia:



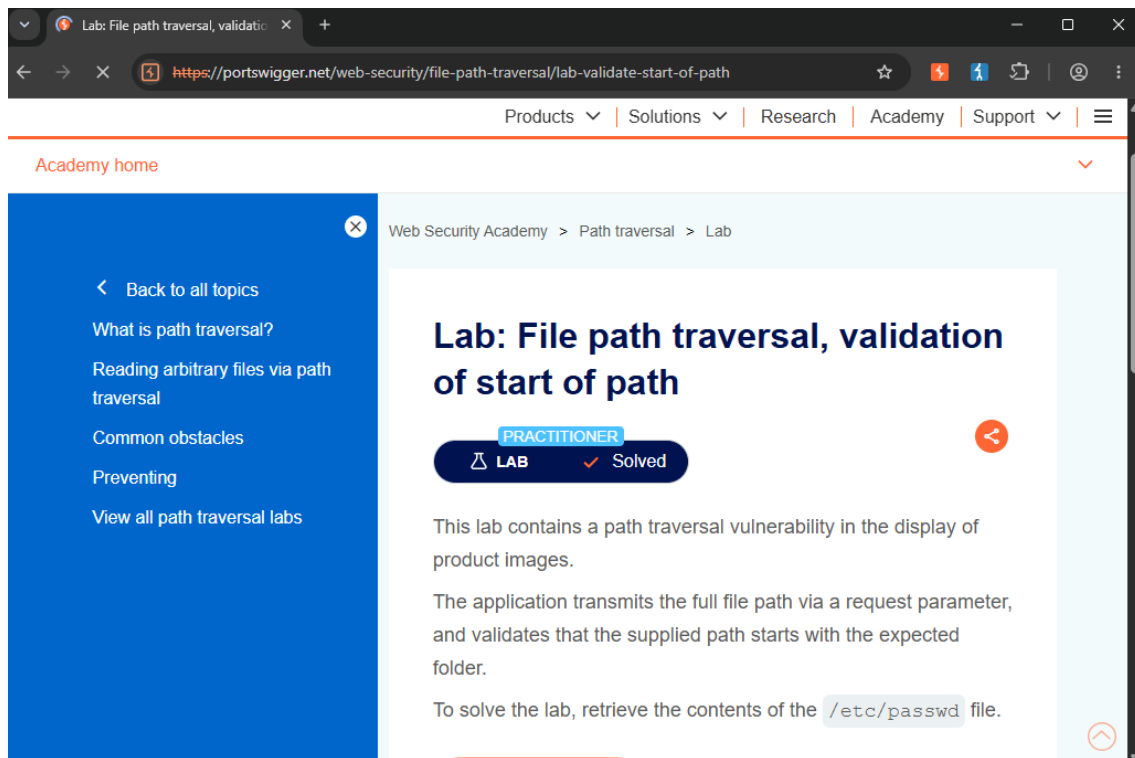
```
Response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System
(admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001:/home/peter:/bin/bash
```

Figura 4. Confirmación de Lab Solved

Descripción:

La captura muestra la confirmación proporcionada por PortSwigger Web Security Academy después de ejecutar correctamente la explotación. El estado **Lab Solved** confirma que el acceso al archivo `/etc/passwd` fue reconocido por la plataforma y que el objetivo establecido para el laboratorio fue alcanzado.

Evidencia:



7. EXPLICACIÓN DEL PAYLOAD

El payload utilizado fue:

`/var/www/images/../../etc/passwd`

El payload puede dividirse en tres partes principales.

7.1. Directorio permitido

`/var/www/images/`

Esta parte corresponde al directorio esperado por la aplicación y permite que la ruta cumpla inicialmente con la validación que comprueba el inicio de la cadena.

7.2. Secuencias de recorrido

`../../..`

Cada secuencia `../` representa un desplazamiento hacia el directorio padre.

Al utilizar tres secuencias consecutivas se puede retroceder dentro de la estructura de directorios hasta salir del directorio destinado a las imágenes.

7.3. Archivo objetivo

`/etc/passwd`

Finalmente, se especifica el archivo que se desea recuperar.

En conjunto, el payload permite que la ruta comience con el prefijo esperado por la aplicación y posteriormente utilice secuencias de recorrido para acceder a un archivo ubicado fuera del directorio autorizado.

El bypass funciona porque la validación se realiza sobre el inicio de la cadena proporcionada, mientras que la ruta posteriormente puede ser interpretada y normalizada por el sistema de archivos.

8. MITIGACIÓN RECOMENDADA

Para prevenir vulnerabilidades de **File Path Traversal**, la aplicación debe evitar confiar directamente en rutas proporcionadas por el usuario.

Una medida importante consiste en **normalizar y resolver la ruta antes de realizar la validación**, verificando posteriormente que la ruta final permanezca dentro del directorio autorizado.

También se recomienda:

- Evitar que el usuario proporcione directamente rutas de archivos.
- Utilizar identificadores internos para seleccionar archivos.
- Normalizar las rutas antes de realizar cualquier validación.
- Verificar que la ruta final resuelta permanezca dentro del directorio permitido.
- Aplicar controles de acceso adecuados sobre los archivos del sistema.
- Ejecutar la aplicación utilizando el principio de mínimo privilegio.
- Evitar que las entradas controladas por el usuario determinen directamente las rutas de archivos que serán procesadas por el servidor.

La validación debe realizarse sobre la **ruta final normalizada**, y no únicamente sobre el texto inicial proporcionado mediante el parámetro filename.

CONCLUSIÓN

El laboratorio permitió comprobar de manera práctica el funcionamiento de una vulnerabilidad de **File Path Traversal** causada por una validación insuficiente del inicio de una ruta.

Mediante el uso del payload `/var/www/images/../../etc/passwd`, fue posible mantener el prefijo esperado por la aplicación y posteriormente utilizar secuencias de recorrido para abandonar el directorio autorizado y acceder al archivo `/etc/passwd`.

La respuesta obtenida por parte del servidor y la posterior confirmación de **Lab Solved** demostraron que la explotación se realizó correctamente.

Este laboratorio permitió comprender que una validación basada únicamente en comprobar el inicio de una ruta no es suficiente para proteger los recursos del sistema. Para evitar este tipo de vulnerabilidades es necesario normalizar y validar la ruta final antes de utilizarla, además de aplicar controles de acceso y el principio de mínimo privilegio.